

Aufgabe 1: Transformers

Teilnahme-ID: 79418

Bearbeiter/-in dieser Aufgabe:
Fabian Schwarz

March 31, 2026

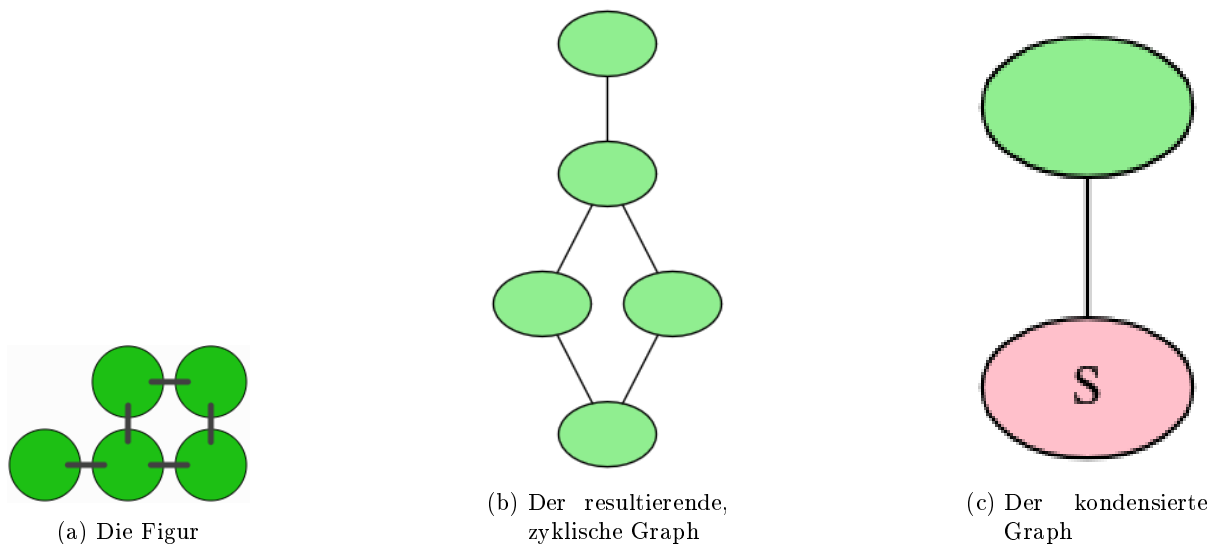
Contents

1	Lösungsidee	1
2	Umsetzung	3
3	Laufzeit	3
4	Werkzeuge	3
5	Beispiele	4
6	Quellcode	4

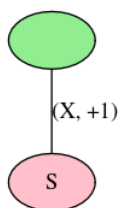
1 Lösungsidee

Man modelliert jede Figur als einen ungerichteten, zusammenhängenden Graphen $G = (V, E)$. Die Knotenmenge V besteht hier aus den Steckplättchen, und die Kantenmenge E aus den Stiften. Jeder Stift $e \in E$ ist hier aus einer Achse (horizontal bzw. vertikal) und einer Richtung (-1 bzw. +1) zusammengesetzt.

Man bemerkt, dass Stifte, die Teil eines Zyklus sind, nicht gedreht werden können, da die Figur sonst "zerreißt". Man kondensiert den Graphen also zu einem neuen Graphen G' , in dem alle Knoten, die durch Zyklen verbunden sind, zu einem Superknoten (S) zusammengefasst sind. Dies sieht beispielsweise so aus:

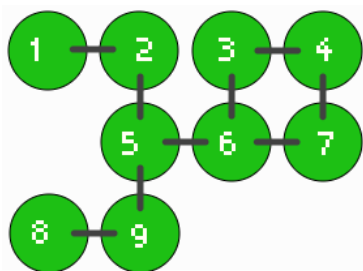


Schließlich markiert man die Stifte mit ihren entsprechenden Richtungen:

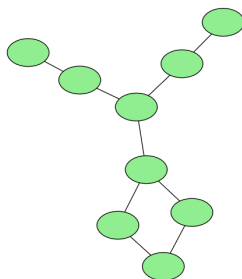


Der Stift ist horizontal, daher X , und verläuft von dem Ursprungsknoten aus nach rechts, daher $+1$. Man stellt hier fest, dass nach Beseitigung aller Zyklen der Graph, da er ja bereits zusammenhängend ist, einen Baum darstellt.

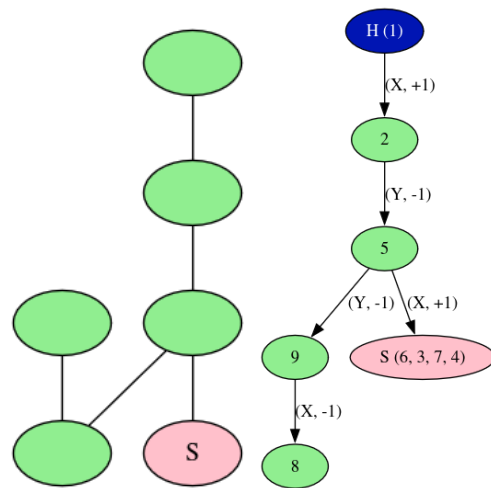
Der Ursprungsknoten kann und muss sogar arbiträr ausgewählt werden. Man hat aktuell noch keinerlei Wissen darüber, welcher Knoten welchem Gegenknoten im anderen Graphen entspricht. Es wird später erläutert, wieso das erlaubt ist. Ein komplexeres Beispiel könnte wie folgt aussehen:



(a) Die Figur



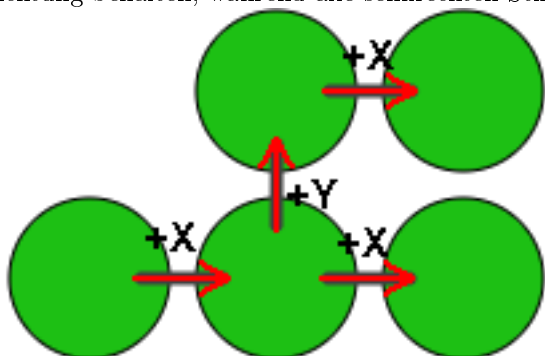
(b) Der resultierende, zyklische Graph



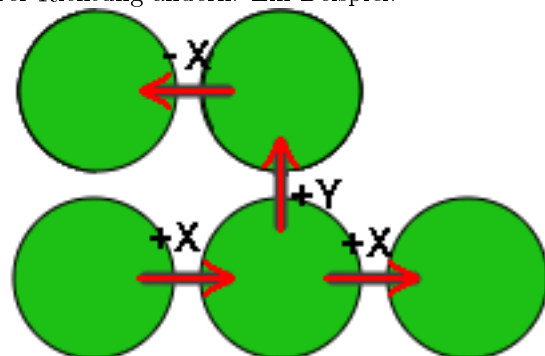
(c) Der kondensierte (d) Der Graph mit Richtungsangaben

Zu Beginn werden den Plättchen beliebige Indices verliehen. Diese bleiben bei der Umwandlung in den Graphen natürlich erhalten, sind nur hier nicht abgebildet. Es werden erneut Zyklen zu Superblöcken kondensiert, und schließlich ein arbiträrer Ursprungsknoten (H) gewählt, von welchem aus der Graph traversiert und die Verbindungen beschriftet werden. Der Graph bleibt ungerichtet, ist hier aber, um das Traversieren zu kennzeichnen, mit Pfeilen dargestellt. Im Superknoten ist gespeichert, aus welchen Knoten dieser zusammengesetzt ist.

Be einer 180° -Drehung um einen Stift beobachtet man, dass alle Stifte parallel zur Drehachse ihre Richtung behalten, während alle senkrechten Stifte das Vorzeichen ihrer Richtung ändern. Ein Beispiel:



Hier wird um den $+Y$ -Stift gedreht.



Aus diesen Bedingungen lassen sich also Regeln für die Überführbarkeit zweier Figuren in einander anhand ihrer kondensierten Bäume bestimmen:

- Die Bäume A und B müssen isomorph sein, das heißt, die gleiche Anzahl an Knoten und Kanten haben, sowie Kanten auf den gleichen Achsen zwischen entsprechenden Knoten. Besteht der

eine Baum also ausgehend von seiner Wurzel aus $V_1 = (+X, +Y, -X)$, so muss V_2 auch eine $(\pm X, \pm Y, \pm X)$ Abfolge sein.

- Zyklen müssen in beiden Figuren identisch sein, d.h., die gleiche Länge und die gleiche Abfolge von X- und Y-Verbindungen besitzen; die genaue Richtung (± 1) ist erstmal irrelevant.
- In Baum B muss es genau ein Plättchen P geben, dass als Drehpunkt fungiert. An ihm ist eine Drehachse X oder Y definiert. Alle Stifte, die über diese Drehachse an P hängen, behalten ihre Richtung bei. Alle Stifte, die über die andere Achse an P hängen, folgen den oben festgestellten Regeln: laufen sie parallel zur Drehachse, behalten sie ihre Richtung identisch bei; andernfalls ändert sich das Vorzeichen der Richtung.
- Zyklen müssen entweder vollständig identisch bleiben, d.h., identische Abfolgen an Verbindungen besitzen, oder, wenn sie gedreht wurden, nach der oben genannten Inversionsregel übereinstimmen. Beispielsweise kann $(+Y, +X, -Y, -X)$ nur identisch bleiben, zu $(-Y, +X, +Y, -X)$ oder zu $(+Y, -X, -Y, +X)$ werden. Diese Abfolgen haben natürlich keinen festen Startpunkt, d.h. $(+Y, +X, -Y, -X) = (-X, +Y, +X, -Y)$.

2 Umsetzung

Zu Beginn werden die Figuren eingelesen und als ungerichtete, gewichtete Graphen dargestellt. Die Gewichte sind hier die Richtung, bestehen also aus `axis` $\in X, Y$ und `sign` $\in -1, +1$.

Um nun Zyklen zu erkennen, werden die Graphen mithilfe einer Tiefensuche in zweifach zusammenhängende Komponenten (*Biconnected Components* nach Hopcroft and Tarjan (1973)¹) geteilt. Artikulationspunkte sind mögliche Drehpunkte unserer Figur, die resultierenden “Blöcke” die starren Zyklen. Hierfür werden während der DFS alle Kanten auf einen Stack gelegt und, sobald ein Artikulationspunkt gefunden wird, alle Plättchen bis zu diesem Punkt als starrer Block betrachtet.

Ein Artikulationspunkt ermittelt sich wie folgt:

1. Man traversiere den Graphen wie gewöhnlich in einer DFS, wobei alle traversierten Kanten auf einen Stack gelegt werden.
2. Jedem Knoten wird eine Entdeckungszeit (en. *discovery time*) `disc` zugewiesen, die für jeden Ablaufschritt inkrementiert wird. Ein zweites Feld `low` wird ebenfalls auf die Entdeckungszeit gesetzt.
3. Während jeder Backtracking-Phase betrachte man:
 - a) für jeden Knoten u , rückwärts gehend, all seine (bereits besuchten) Nachbarn v_i , bis auf seinen direkten Vorgänger. Die von allen Nachbarn v_i geringste `disc` übernimmt der Knoten u als `low`.
 - b) für jede Kante $u \rightarrow v$, rückwärts gehend², ob $v.\text{low} \geq u.\text{disc}$. Falls eine Gleichheit vorliegt, handelt es sich um einen Artikulationspunkt. Alle Knoten, die seit Beginn des backward-pass abgelaufen wurden, werden in den Artikulationspunkt als Superknoten vereint.³ Wenn der `low`-Wert echt größer ist, ist diese Verbindung eine Brücke, d.h. ein drehbarer Stift, und wird im resultierenden Baum zu einer Kante zwischen zwei (Super-)Knoten.

Hiermit zerfällt die Figur in eine Menge von Superknoten, Artikulationspunkten (gewöhnlichen Plättchen) und Brücken, die sich zu dem kondensierten Baum zusammenfassen lassen (dem sog. *block-cut tree*).

3 Laufzeit

4 Werkzeuge

Der Code wurde vollständig in Rust 1.94.1 (e408947bf 2026-03-25) geschrieben.

¹Hopcroft, J., Tarjan, R. (1973). Algorithm 447: efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6), 372–378. <https://doi.org/10.1145/362248.362272>

²Im forward-pass wurde also eine Kante $u \rightarrow v$ abgelaufen; eigentlich läuft man hier von $v \rightarrow u$ rückwärts, dennoch wird die Kante hier als $u \rightarrow v$ beschrieben, wobei u die “ältere” ist.

³Der Artikulationspunkt ist der Punkt, der den Block mit dem Rest des Graphen verbindet. Die Wurzel der DFS ist nur dann ein Artikulationspunkt, wenn sie im DFS-Baum mehr als ein Kind hat.

5 Beispiele

```
1  
2  
3
```

6 Quellcode

Auslassungen sind hier mit `/* [...] */` markiert.

```
fn solve() {  
    return;  
}
```
